FruitGame: design & build reference

Snapshot at milestone M3 (face control complete), 2026-10-27

A 2.5D webcam minigame. The player's own (background-removed) webcam image stands in the scene. A basket follows their **face left and right**, and they catch fruit falling **one at a time** from the treetops. Each caught fruit joins a growing pile behind them.

This document is the reference for building the game. Points from the handwritten plan (`unity plan.pdf`) are marked **[plan]**. Answers to the first round of open questions are marked **[decided]** (see §14). Everything else is a decision made here to fill gaps, marked **[filled in]**, and can be changed. Numbers in the tunables table are starting values to tweak during playtesting.

---

1. At a glance

Engine	Unity 6000.6.7f1, URP 17.6, Input System 1.20
ML runtime	Unity Inference Engine (`com.unity.ai.inference` 2.6.1, already installed)
Target	**macOS and Windows** standalone (runs in the Editor too). Everything chosen here (WebCamTexture, Inference Engine, URP) is cross-platform; anything that turns out macOS-only gets a Windows fallback or is flagged.
Screen	**Portrait 3:4 [plan]**, reference 1080 × 1440, windowed
Control	Face x position from the webcam **[plan]**; keyboard/mouse fallback for testing **[filled in]**
Style	Low-poly, flat-shaded, bright and minimalist **[plan]**
Session	**60 s timed round [decided]**. A lives counter is coded in but switched off, as a fallback.
Audience	**Accessibility first [decided]**: per-player calibration of head movement, so even a small comfortable range controls the full basket travel

---

2. Scene layers (front to back)

The plan stacks the scene as parallel planes along Z. The camera is a **perspective camera with a narrow FOV**, so the depth difference gives a gentle 2.5D parallax without looking like a full 3D game.

Layer	What	Z (world)	Notes
L0	Score counter **[plan]** + round timer	UI, Screen Space - Overlay, inside a "Frame" rect fitted to the 3:4 viewport	Cartoon, minimalist, 2.5D stylised **[plan]**. Score top-left, per the sketch; timer top-right in the same style.

Layer	What	Z (world)	Notes
L1	Foreground **[plan]**	⊠1.1 ⋯ ⊠0.6	Low-poly trees and shrubbery **[plan]**. **Four small trees** (two pines, a cypress, a small oak) toward the bottom corners **[decided]**, with recoloured Kenney bushes, grass and flowers along the bottom edge.
L2	Falling fruit + basket **[plan]**	0	The gameplay plane. All gameplay maths happen here, in 2D (x, y).
L3	Webcam person, background removed **[plan]**	2	Unlit textured quad: camera colour × segmentation mask.
L4	Background fruit pile **[plan]**	≈ 3.0 ⋯ 4.2	Caught fruit stacks up with physics, "between webcam person and removed background" **[plan]**. Built ahead of schedule (§7).
L4b	Distant trees **[decided]**	5.2 ⋯ 5.4	Two oak canopies framing the top corners plus two pines, every colour pulled 60–70 % toward a light-blue haze (#A9C6DD) so they read as far away, even against webcam footage. They don't cast or receive shadows.
L5	Webcam background, "image processed bokeh" **[plan]**	6	The same camera frame, heavily blurred, so the real room shows as soft bokeh.

Camera [filled in]: position (0, 0, ⊠12), looking +Z, vertical FOV 30°. `PortraitLetterbox` keeps it at 3:4 in any window size (black bars from a second camera) and stretches the HUD frame to match. At z = 0 the visible area is about **4.8 m wide × 6.4 m tall**; one world unit is one metre. Quads behind z = 0 are scaled by `(12 + z) / 12` so they still fill the frame exactly.

Lighting [filled in]: one warm directional light (about 50° down, from the upper left) with soft shadows, plus a light-blue/soft-green gradient ambient. Fruit and trees use flat-shaded lit materials. The person and bokeh quads are **unlit**, so the webcam image keeps its true colour and doesn't pick up scene shadows.

---

3. Core loop

1. **Boot**: open the webcam and load the ML models.
2. **Calibrate**: three quick poses (see §5). Skipped automatically if a saved calibration exists, but can always be redone.
3. **Countdown**: 3 · 2 · 1.
4. **Playing**: exactly **one fruit falls at a time [plan]**. The next fruit spawns only after the current one has been caught or missed **[plan]**, with a 0.6 s pause in between **[filled in]**.
5. **Game over** when the **60 s timer** reaches 0:00 **[decided]**: show the score, the per-fruit tally and the best score, then play again. A fruit that is mid-air at 0:00 is allowed to land (caught or missed) before the round ends, so the last catch is never stolen.

Rules

- **Caught** = points for that fruit type (all 1 for now) **[decided]**. The fruit pops into the basket, then a copy of it drops onto the background pile (L4).

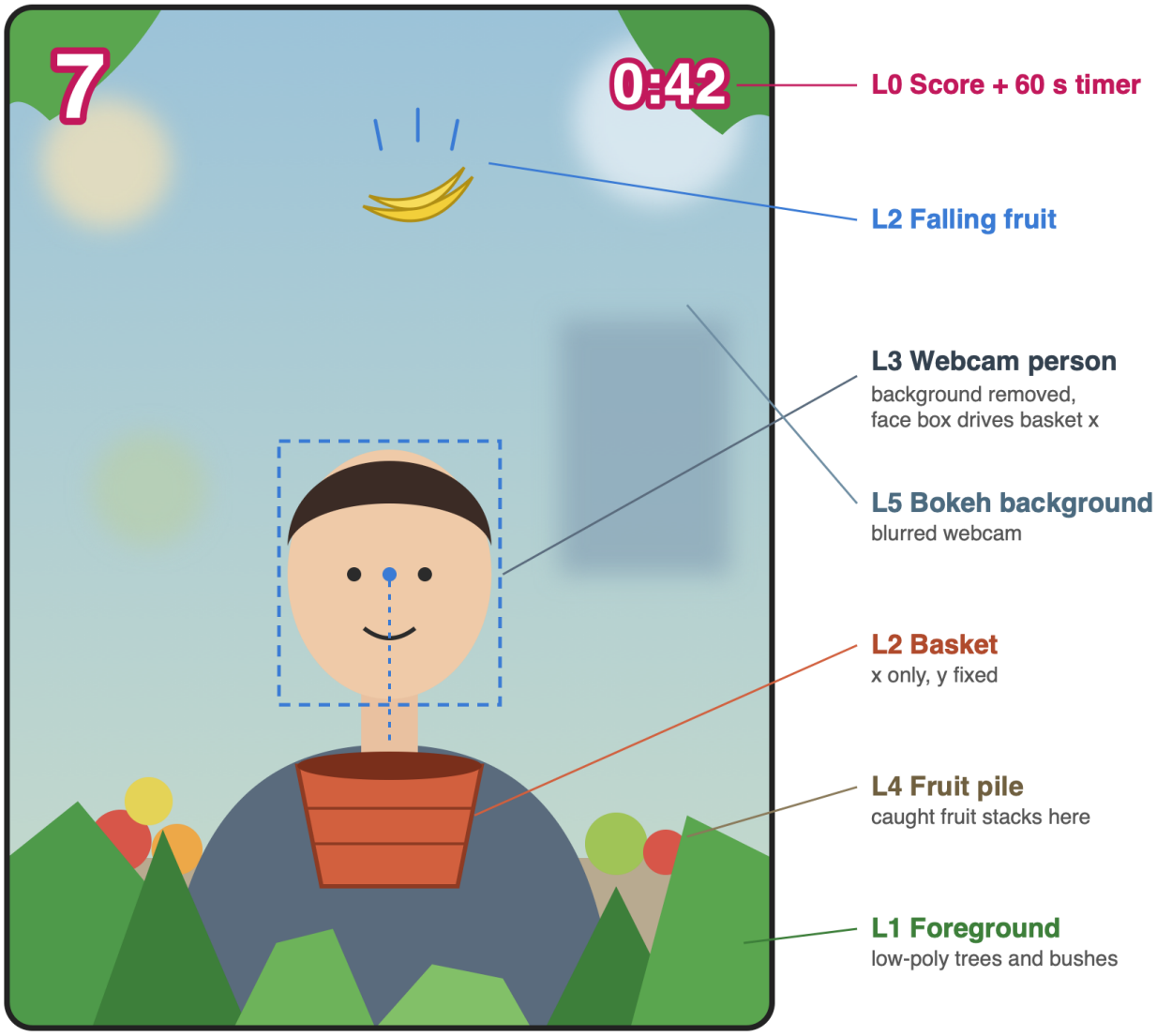
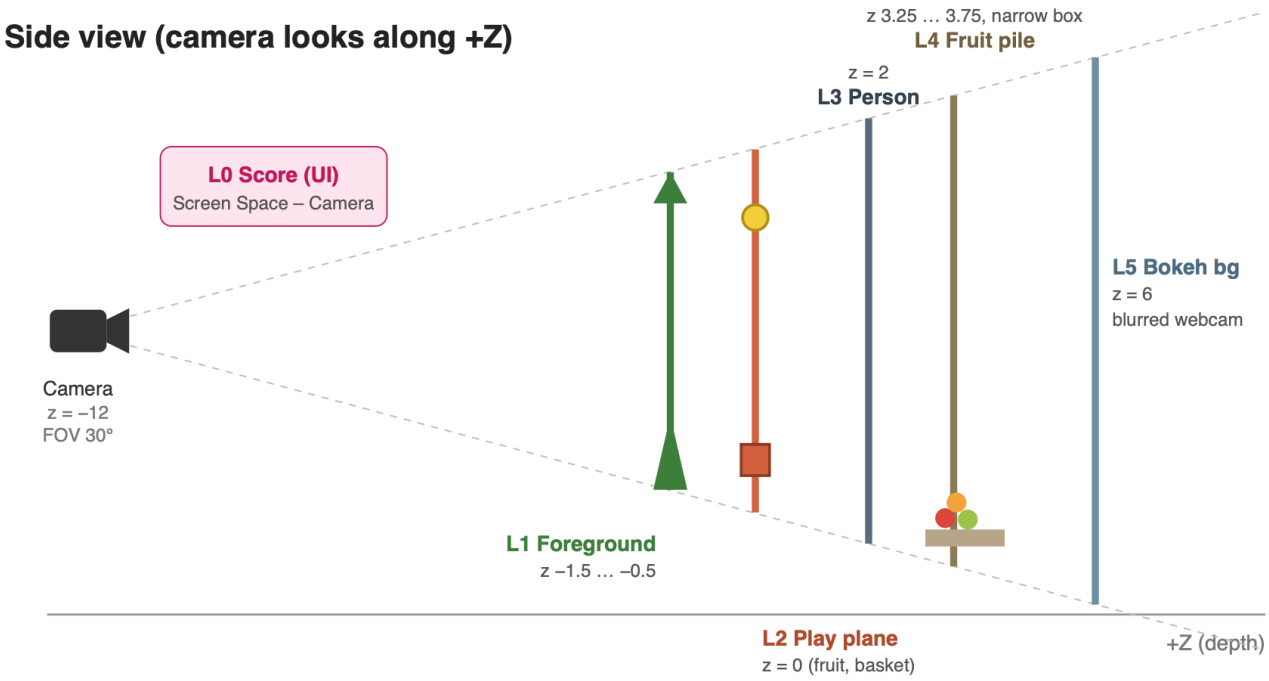


Figure 1: Screen layout



Quads behind z = 0 are scaled by (12 + z) / 12 so they still fill the frame; the slight perspective gives the 2.5D parallax.

Figure 2: Depth layers

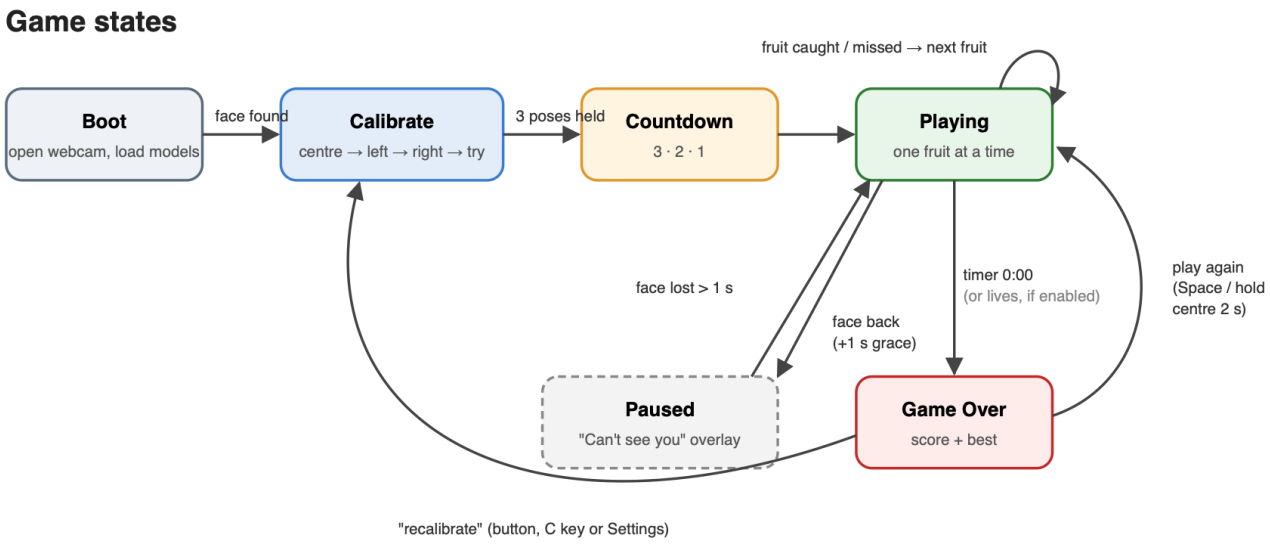


Figure 3: Game states

- **Missed** = no penalty in timed mode; it only costs time. The miss is counted in the stats. The fruit falls off-screen behind the bottom bushes.
- **Round timer [decided]**: 60 s, shown top-right. It pauses whenever the game pauses (face lost, settings open). The last 10 s pulse.
- **Lives (coded, off) [decided]**: a `LivesCounter` component and lives HUD exist and are tested, but `GameConfig.endCondition = Timer` keeps them inactive. Switching to `Lives` (or `TimerOrLives`) makes 3 misses end the round, with no other code change.
- **Difficulty ramp [filled in]**: driven by elapsed round time. Fall speed goes from 2.7 to 3.8 m/s and the drift cap from 0.7 to 1.2 m/s along an ease-in curve over the 60 s. It stays easy on purpose: the plan says the drift is "capped to make game easy" **[plan]**.
- **Best score** is saved in PlayerPrefs.

Fruit types & scoring hooks [decided]

Every fruit is its own type, even though they're all worth the same for now, so variety-based scoring can be added later without a refactor:

- A `FruitType` ScriptableObject per fruit (Apple, Banana, Orange, Pear, Lemon, Mango, Grapefruit, Pineapple, Melon) holds its id, display name, prefab(s), points (= 1), spawn weight, collider shape and icon.
- Score goes through one `ScoreRules.PointsFor(FruitType, context)` function. Today it returns `type.points`; later it can apply combos, streaks or rarity.
- `RoundStats` tracks caught and missed counts **per type**, and the Game Over screen shows them as a small icon tally.

---

4. Falling fruit

Property	Spec
Spawn position	Random x in the band ±2.1 m, at y = +3.6 (just above the top edge, so fruit appears to drop out of the canopy) **[plan: randomised falling position]**
Vertical speed	**Constant [plan]**: 2.7 m/s at the start, ramping to at most 3.8 m/s (raised after playtest 1)
Sideways speed	**Random but capped [plan]**: |vx| ≤ 0.7 m/s at the start, up to 1.2 m/s, wide enough that wall bounces are common and visible (widened after playtest 1)
Walls	Invisible walls at x = ±2.25 reflect vx (in both phases below), so fruit never leaves the screen sideways **[filled in]**
Spin	**Random rotational velocity [plan]**: 120–240 °/s around a random axis, fast enough that you can read the fruit's shape as it falls (was 15–45 °/s) **[decided]**. Cosmetic: the fall itself is scripted, and spin only matters once a fruit bounces.
Size	Every fruit is normalised to about **0.88 m** across (`fruitSize` × the type's `sizeScale`), whatever its model scale (0.45 m in M1, 0.55 m in M2, ×1.6 on request) **[decided]**
Types	Apple, Banana, Orange, Pear, Lemon, Mango, Grapefruit (common); Pineapple and Melon (rarer and a bit bigger). All worth 1 point for now.

Two-phase movement with real basket collisions [decided after playtest 1]

Each fruit is a Rigidbody with a sphere collider, locked to z = 0.

1. **Scripted fall**: every physics step its velocity is held at `(vx, −vy)`, so the "constant downward speed" stays exact. The assist (§6) nudges vx and the walls reflect it.
2. **Knocked**: the moment it touches the basket (rim, side wall or floor edge), the scripted fall stops and physics takes over. The fruit bounces (bounciness 0.45) and falls under `knockedGravity` (9 m/s²). The assist no longer applies, so a basket that

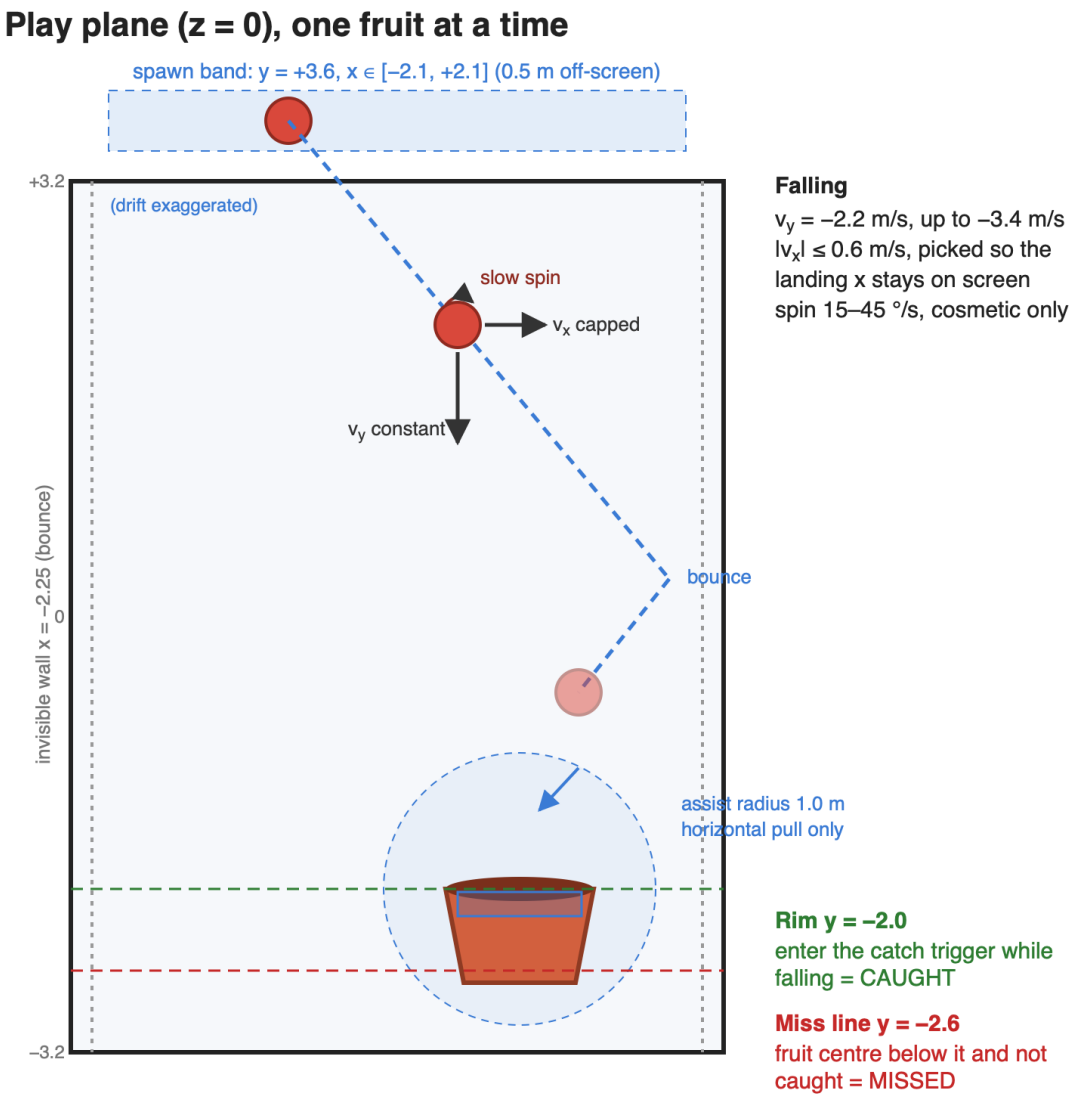


Figure 4: Fruit motion

arrives late bangs the fruit away instead of swallowing it. The basket is a kinematic Rigidbody moved with `MovePosition`, so a fast-moving basket also shoves fruit sideways.

Catch / miss detection

- **Caught**: the fruit's centre is inside the tub (below the rim, between the tapered inner walls), detected by a trigger inside the basket. That covers clean drops and lucky bounces off the rim that end up inside.
- **Missed**: the fruit's centre drops below the **miss line** (y = ⊠2.6) without being caught, or a knocked fruit is still unresolved after 4 s (a safety net in case it balances on the rim).
- **Bump feedback**: every basket contact plays the "tink" and wobbles the basket (sound throttled to once per 0.12 s).
- Because the walls are solid, a fruit (radius ≈ 0.22 m) only drops in cleanly within about ±0.25 m of the basket centre. Nearer the edges it clips the rim and it's up to physics whether it falls in or out. The bigger assist (§6) compensates.

---

5. Basket & face control

Property	Spec
Axis	**Locked to webcam face tracking on the x axis [plan]**; y is fixed with the rim at ⊠2.0 (roughly chest height, under the chin as in the sketch)
Size	Mouth **1.6 m** wide (a third of the screen), 0.8 m tall, scaled up with the ×1.6 fruit so a clean drop still has about ±0.3 m of room. The model tilts 5° toward the camera (was 12°, which looked too obviously slanted).
Travel	x clamped to ±1.45 m so the basket never leaves the screen
Movement	**Scaled**, so the player doesn't have to move their face all the way **[plan]** (see mapping below). The player's own comfortable range maps to the full basket travel **[decided]**.

Mapping, per frame [filled in]

1. `u` = horizontal centre of the tracked face box in the **mirrored** camera image (0⋯1). Mirroring means moving left moves the basket left, like a mirror.
2. `t = (u − u_centre) / (u_centre − u_left)` when the face is left of centre, and `(u − u_centre) / (u_right − u_centre)` when it is right of centre. Each side uses its **own** measured range, so a player who can turn further one way than the other still reaches both edges **[decided]**. The **Sensitivity** setting (0.6×–1.6×) scales on top.
3. Dead zone: `|t| < 0.04 → 0`, so small head jitter at rest doesn't move the basket.
4. `x = limit · softclamp(t)`. This is linear through the middle and eases into ±limit, so overshooting your calibrated range doesn't feel like hitting a wall.
5. A **One-Euro filter** on x: smooth when still, responsive when moving fast.
6. `MoveTowards` with a max speed of 18 m/s (doubled after playtest 1) to absorb tracker glitches.

Note: the basket deliberately moves further than the face does on screen, so it won't sit exactly under the person's chin. That is intended **[decided]**: crossing the whole camera shot with your head is unviable, especially for players with limited neck or upper-body movement.

Calibration stage [plan: "perhaps a pre-game calibration stage later" → decided: core feature]

Calibration measures how far **this** player can comfortably move their head, so they get the finest possible control over the basket **[decided]**. It runs before the first round and is always available from Settings and from the Game Over screen.

1. **"Look straight at the camera"**: a 1 s steady hold records `u_centre`.
2. **"Move your head left as far as is comfortable"**: records `u_left`, the furthest held position (not a momentary peak).
3. **"Move your head right as far as is comfortable"**: records `u_right`.
4. **Try it**: a short practice where the basket follows the new mapping and the player can accept or redo it.

Accessibility details **[decided / filled in]**:

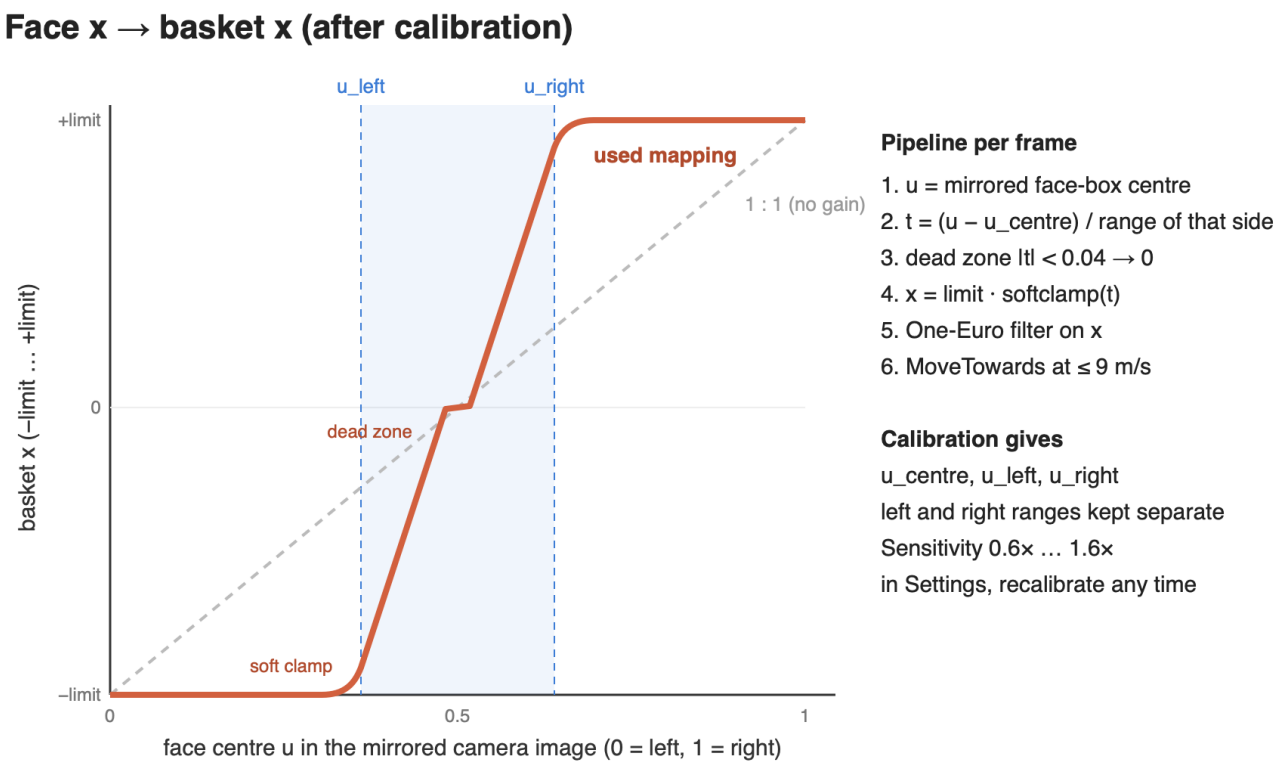


Figure 5: Face to basket mapping

- "Steady" means the face position varies by less than 0.01 over 1 s, shown as a filling ring around the face. The tolerance and hold time are settings, for players with tremor.
- Left and right ranges are stored **separately**, and there is no symmetric `min()`.
- A very small range is accepted; there is only a floor of 0.03 (u units) per side to keep the maths stable. Smaller ranges just mean higher effective sensitivity, and the dead zone and One-Euro filter scale with the range so it stays steady.
- If the face box is under 12 % or over 45 % of the frame width, a prompt says "move closer" or "move back".
- Prompts use large text plus arrows; optional voice/sound cues are a polish item.
- Results are saved in PlayerPrefs, so a returning player can skip straight to playing. Press **C** to recalibrate. Defaults if skipped: 0.5 / 0.35 / 0.65.

Face tracking robustness [filled in]

- With several faces, track the largest box, but stick with the previous target while it still overlaps it (IoU > 0.3).
- Face lost: the basket holds its position. After **1 s** the game **pauses** with a "Can't see you" overlay, then resumes 1 s after the face returns.
- Keyboard ← → / A D and mouse x drive the basket when no webcam is present or when **F2** is toggled (for testing).

---

6. Assist attractor

"Physics slight attractor with capped radius for assist" **[plan]**, specified as **[filled in]**:

- Only acts on the current falling fruit, only during its **scripted fall** (never once it has been knocked by the basket), only while it is **above the rim**, and only within **R = 1.8 m** of the basket mouth centre (raised from 1.0 m after playtest 1).
- It adds a **horizontal** acceleration toward the basket's x: `a = strength · (1 − d/R)`, with a strength of 4 m/s² (was 1.5), and capped so vx never exceeds the drift cap + 1.0 m/s.
- It never touches vertical speed, so the "constant y speed" rule holds.
- It noticeably pulls a near-miss toward the mouth, but a basket that's too late still gets hit and knocks the fruit away. It can be tuned to 0 in settings ("Assist: Off / Low / Normal").

---

7. Background fruit pile

"Collected fruits stack up with collision → should put care so there are no physics glitches" **[plan]**. Placed on L4, between the person and the bokeh **[plan]**.

How it works [decided]: when a fruit is caught, the falling fruit is consumed by the basket (pop + sparkle). A **pile copy** of the same fruit type then drops from above, behind the person, into a **very narrow invisible rectangular container**. The container is only about one fruit deep, so the fruit stacks like a 2D heap but still looks 3D. Nothing stacks inside the basket. Over the round a colourful heap builds up behind the player, partly hidden by the foreground bushes.

The container [decided / filled in]

- Invisible box colliders: a floor (y = ☒3.6) on a ground strip, left and right walls at x = ±3.0 (near the screen edges at that depth), and **front and back walls `pileDepthFactor` (1.15) × the largest fruit's diameter apart**. That's ≈ 1.27 m now that fruit are 0.88 m, so two fruit can never sit side by side in depth.
- Built by `FruitPile` at runtime from `GameConfig`; caught fruit are added by `GameStateMachine`, and the pile is cleared at the start of each round and on Main menu.
- Elongated fruit (banana, pear, pineapple) get a capsule fitted along their longest axis; the rest get spheres.
- **Background look [decided 2026-10-27]:** pile fruit must never be mistaken for a falling fruit. They use `Fruit_Atlas_Far`, a copy of the fruit atlas with the same light-blue haze as the distant trees baked in (`FruitAtlas_Far.png`, made with `magick FruitAtlas.png -fill "#A9C6DD" -colorize 65%`). They neither cast nor receive shadows. Falling fruit keep the full-colour atlas.
- Stress-tested 2026-10-27: 50 fruit dropped in bursts. It capped at 40, 36/40 were frozen once settled, none escaped the container, and the heap reached y ≈ ☒0.9. At 0.88 m a full pile is tall and busy behind the falling fruit; lower `pileMax` if it competes for attention.
- Fruit keep full 3D rotation and tumble naturally between the walls, which is what makes the pile read as 3D rather than flat sprites.

- Drop point: a random x across the container, from y = +3.6, so the copy falls through the frame behind the person.

Anti-glitch measures [filled in]

Measure	Why
Simple colliders: sphere for round fruit, capsule for banana and pear, never mesh colliders	Stable stacking, cheap
Narrow front/back walls instead of a frozen Z axis; optionally a soft Z-centring force if a fruit jams diagonally	Keeps the 2D heap while rotation stays free; nothing tumbles toward the camera
Interpolate, collision mode **Continuous Dynamic** `maxDepenetrationVelocity = 1.5`	No tunnelling or jitter at render rate Overlaps resolve gently instead of fruit "exploding" out
Spawn only where `Physics.CheckSphere` finds space; otherwise retry another x up to 5 times, then spawn higher	Never spawn inside another fruit
Physics material: friction 0.8, bounciness 0.05	Settles quickly
Freeze when settled: speed < 0.05 m/s for 1.5 s → `isKinematic = true`	The pile stops simulating; a big pile costs nothing and can't creep
Cap of **40 pile fruits**; past that the oldest shrinks to nothing over 0.3 s, and every frozen fruit is woken so the heap settles into the gap	Bounded cost, no floating fruit
Its own physics layer `Pile` (layer 8, for both fruit and container walls), set to ignore every other layer at runtime	Can't interact with gameplay fruit or the basket
Solver iterations 10 / velocity iterations 4 for the pile bodies; fixed timestep 1/60	Better stacking stability

---

8. Webcam pipeline

Stage	Spec **[filled in]**
Capture	`WebCamTexture` with 1280×720 at 30 fps requested; device selectable in settings
Prep	A blit mirrors it and centre-crops to 3:4 into a 540×720 RenderTexture
Face detection	**BlazeFace** (short-range) through Unity Inference Engine, 128×128 input, GPU backend. Unity publishes a ready-made version on Hugging Face (`unity/inference-engine-blaze-face`); **check the exact repo name and model format at milestone M3**. Runs every camera frame, with async readback so it never stalls rendering.
Person segmentation	**MediaPipe Selfie Segmentation** (256×256 square variant), ONNX or LiteRT, through Inference Engine. Gives a 0…1 person mask. **Check model availability and licence (Apache-2.0) at M4.**
Mask cleanup	1-pass blur for soft edges, a small erode against haloing, and a temporal lerp (0.6) to stop flicker
Person quad (L3)	Unlit shader: `rgb = cam`, `alpha = smoothstep(0.4, 0.6, mask)`
Bokeh quad (L5)	Unlit shader on a downsampled (1/4) camera frame: two-pass Gaussian or Kawase blur at a large radius, 15 % desaturation, slight brightening and a light tint toward the scene palette. Optional: a few soft circular highlights from the brightest areas for a stronger "bokeh" look.

Stage	Spec **[filled in]**
Fallback	If segmentation fails to load: the person quad shows the full frame and the bokeh layer is hidden. If there's no camera at all: the bokeh layer becomes a static soft gradient, the person layer is hidden, and the keyboard controls the basket.

Cross-platform notes [decided: macOS + Windows]

- WebCamTexture, Inference Engine (GPU compute: Metal on macOS, DX12/Vulkan on Windows) and URP all run on both platforms. Use the GPU backend, falling back to CPU if the GPU backend fails.
- macOS needs camera permission: set **Player Settings → Camera Usage Description**, and allow Unity/the built app under System Settings → Privacy → Camera.
- Windows needs no player setting, but the camera can be blocked under Settings → Privacy → Camera; show a clear message if `WebCamTexture` produces no frames.
- Webcams report different resolutions and orientations (`videoRotationAngle`, `videoVerticallyMirrored`); the prep blit handles both.
- Shaders are HLSL/ShaderGraph only, with no platform-specific code paths.

---

9. UI & feedback

Score counter (L0) [plan: cartoon minimalist 2.5D stylised → filled in]

- Top-left, as in the sketch. A big rounded heavy font (e.g. Fredoka or Baloo 2, both OFL) in TextMeshPro: white fill, thick magenta outline (#C2185B), and a hard drop shadow offset down-right so it reads like an extruded sticker.
- On +1: scale-pop 1 → 1.25 → 1 over 0.2 s, plus a small "+1" that floats up from the basket.
- **Timer**: top-right in the same sticker style (`0:42`), pulsing during the last 10 s.
- Lives: three small fruit icons under the score; hidden while lives are off.

Depth cues: keeping the play layer readable [decided 2026-10-27]

- **Rim light on live fruit:** falling fruit use `Fruit_Atlas_Live` with the custom `FruitGame/Fruit Rim` shader (simple lit + fresnel rim, `_RimStrength` 0.7, `_RimPower` 2.5). Shadows and depth are borrowed from URP Lit, so the fruit still casts shadows and feeds depth of field. Pile fruit keep their hazed material.
- Ground shadow under the falling fruit (removed): tried (a soft ellipse at rim height with a light halo) and **removed at your request**; it didn't fit the look.
- **Far-layer blur:** Gaussian depth of field in `GamePostProfile`, from 14.5 m to 18 m from the camera (max radius 1). The play plane (12 m) and the person (14 m) stay sharp, the pile (~15.6 m) softens slightly, and the distant trees (~17 m) soften more.
- Rejected as hurting the look: a smaller pile, a lower pile cap, and the ground shadow.

Juice [filled in]: a catch squashes the basket vertically for 0.12 s and bursts a few leaf-coloured particles. A miss plays a soft thud and a brief red tint on the lost life icon. A rustle sound plays from the canopy as each fruit spawns, to telegraph it.

Audio [filled in]: pop (catch), tink (rim graze), thud (miss), rustle (spawn), and a gentle looping ambience. All short, one mixer, nothing loud.

Screens: fully mouse-operable for a booth [decided after playtest 1]. Nobody should need to look at the keyboard; keys stay as shortcuts only. All buttons are large (≥130 px in the 1080×1440 frame) rounded rectangles in pink (primary) or slate (secondary).

Screen	Contents	Keyboard shortcut
Start menu	"Catch the fruit!", a hint, big **Play** button	Space / Enter
Playing	Score, timer, **II** pause button top-centre	P / Esc to pause
Pause menu	Dimmed game, **Resume**, **Main menu** (abandons the round)	P / Esc / Space to resume
Game over	"Time's up!", score (same size as before), best, **fruit tally in two small columns**, missed count, **Play again**, **Main menu**	Space = play again

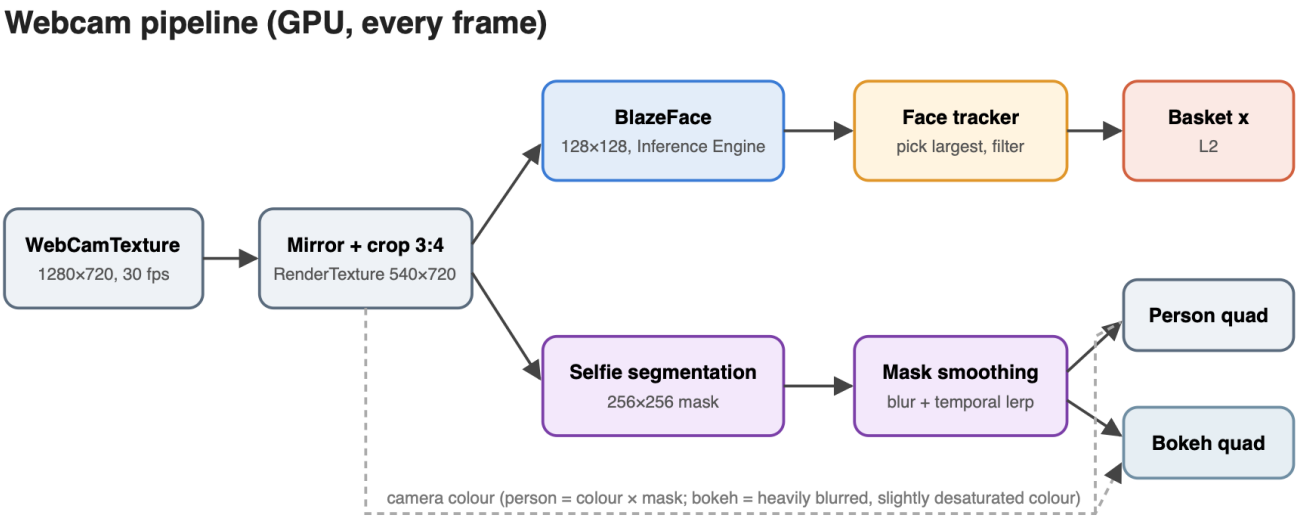


Figure 6: Webcam pipeline

Planned: the calibration overlay (prompts plus a steady-ring) and a small Settings panel (sensitivity, recalibrate, steady-hold tolerance and time, assist, camera device, mirror), both mouse-operable. A debug overlay on **F1** shows the face box, the mask, the mapping value t and FPS.

---

10. Art & assets

Approved downloads (2026-10-26) → `~/Documents/3D Assets/`

Folder	Contents
`Kenney Nature Kit (recoloured)/`	Nature Kit 2.1 OBJ models (CC0). Greens remapped to the tree pack (grass #68AB42, leafsGreen #5A9522, leafsDark #46741A) and stones to warm grey; originals are listed in its README.txt
`FruitGame Extras/Audio/`	The first pick of each sound group: catch_pop, rim_tink, miss_thud, spawn_rustle, pile_land, countdown_tick, round_over (Kenney, CC0; mapping in CREDITS.txt)
`FruitGame Extras/Fonts/`	Fredoka variable font + `Fredoka-Bold.ttf` (a static 700 instance made with fontTools), OFL
`Downloads/`	Untouched original downloads + `_previews/`

The sounds and Fredoka-Bold are already imported into `Assets/FruitGame/`; the Nature Kit gets imported in M2.

Provided packs (`~/Documents/3D Assets/`)

Pack	Use
Low Poly Food (FBX / GLTF / OBJ, gradient-atlas `Texture.png`)	`Fruits/*/<Name>_01.fbx` and `_02.fbx` (two variants each) for falling and pile fruit. Slices are unused (possible future "splat" on miss).
Low Poly Trees (Lite): Bare, Pine, Oak, Maple, Cypress	Oak and Maple for the canopy in the top corners; Pine and Cypress partly visible at the side edges. The licence forbids redistribution, so don't push them to a public repo.

Gaps to fill → downloaded or made into `~/Documents/3D Assets/`, shown for a yes/no, then imported [decided]

Downloaded packs go into the asset folder first and are previewed straight away. Only approved items get imported into the project.

Asset	Plan
Basket (done)	Modelled ourselves: `FruitGame Extras/Basket/make_basket.py` generates `Basket.obj/.mtl`, a flat-shaded 10-sided tapered tub with a chunky rim and two bands in alternating wicker terracottas (#D2603E / #C4552F, bands #8E3B23, inside #7A2F1B). In game it's tilted 12° toward the camera so the mouth reads, and scaled to `basketMouthWidth`/`basketHeight` so it lines up with the physics walls.
Bushes / shrubbery, grass, rocks (done)	Kenney Nature Kit (CC0), recoloured: bushes, grass, flowers, a mushroom group and a rock along the bottom edge
Pile floor	A simple low-poly ground strip in muted earth tones, mostly hidden behind the foreground
Font	Fredoka (Google Fonts, OFL)
SFX	Kenney audio packs (CC0): pop, tink, thud, rustle

Palette: leaf greens #3D7F3A / #5AA64F / #82C068, basket #D2603E / #8E3B23, score #C2185B, sky tint #9CC3D9. Fruit keep their own atlas colours.

---

11. Tunables (`GameConfig` ScriptableObject)

Key	Default	Range
`endCondition`	`Timer`	`Timer` / `Lives` / `TimerOrLives`
`roundDuration`	60 s	
`fallSpeedStart` / `fallSpeedMax`	2.7 / 3.8 m/s	eased over the round
`driftCapStart` / `driftCapMax`	0.7 / 1.2 m/s	
`spinDegPerSec`	120–240	
`spawnXRange`	±2.1 m	
`nextFruitDelay`	0.6 s	
`lives`	3	used only when the end condition includes lives
`basketMouthWidth` / `basketHeight` / `basketTaper`	1.6 m / 0.8 m / 0.75	the colliders are built from these, and the model is scaled to match
`fruitSize`	0.88 m	
`basketXLimit`	1.45 m	
`rimY` / `missY`	☒2.0 / ☒2.6	
`sensitivity`	1.0	0.6–1.6
`deadZone`	0.04	in t units, so it scales with the calibrated range
`calibMinRangePerSide`	0.03	
`steadyTolerance` / `steadyHoldTime`	0.01 / 1.0 s	adjustable for accessibility
`pileZ` / `pileDepthFactor`	3.6 / 1.15 × largest fruit	
`pileFloorY` / `pileHalfWidth` / `pileSpawnY`	☒3.6 / 3.0 / 3.6	
`oneEuroMinCutoff` / `oneEuroBeta`	1.2 / 0.4	basket x in metres
`softClampKnee`	0.8	
`faceReturnGrace`	1.0 s	
`basketMaxSpeed`	18 m/s	
`knockedGravity` / `fruitBounciness` / `fruitFriction`	9 m/s² / 0.45 / 0.3	physics after a basket hit
`knockedTimeout`	4 s	
`assistRadius` / `assistStrength`	1.8 m / 4 m/s²	0 = off
`faceLostPauseDelay`	1.0 s	
`pileMax`	40	
`pileFreezeSpeed` / `pileFreezeTime`	0.05 m/s / 1.5 s	
`bokehBlurRadius` / `bokehDesaturate`	large / 0.15	

---

12. Project layout & scripts

Files marked (done) exist (M1); the rest are planned.

```
Assets/FruitGame/
  Art/      Fonts/ (done) (Fredoka-Bold + SDF asset + Sticker material)  Materials/ (done) (greybox + Palette/)  F
          Models/ (done) Fruits/ (9 × _01/_02 FBX + FruitAtlas.png)  Trees/ (Oak, Pine, Cypress)  Nature/ (recolo
  Prefabs/    Art/ (done) (generated: Fruits/, Trees/, Nature/, Basket/)
  Audio/      (done) 7 clips
  Config/     GameConfig.asset (done)  FruitTypes/<Name>.asset (done) (9 types)
  ML/         blazeface.(onnx|sentis)  selfie_segmentation.(onnx|tflite)
```

```
Prefabs/    Fruit_<Type>.prefab  PileFruit_<Type>.prefab  Basket.prefab  Bush_*.prefab
Scenes/     Game.unity (done) (generated)
Shaders/    FruitRim.shader (done)  PersonCutout.shader  Bokeh.shader  MaskBlur.compute
Scripts/
 Core/    GameConfig (done), GameStateMachine (done), SaveData (done), RoundTimer (done), LivesCounter (done
 Webcam/   WebcamSource (done), WebcamBackdrop (done) (temporary until M4), FaceTracker (done), PersonSegment
 Control/  BasketInputSource (done), FallbackInput (done), BasketInputRouter (done), FaceToBasketMapper (don
             CalibrationData (done), CalibrationController (done)
 Gameplay/  FruitType (done), ScoreRules (done), RoundStats (done), FruitSpawner (done), FallingFruit (done),
             BasketController (done), BasketAssist (done), FruitPile (done), PileFruit (done)
 UI/     GameHUD (done) (score, timer, lives, messages, game over), FaceDebugOverlay (done) (F1), Calibration
  Feedback/  AudioFeedback (done), JuiceFx
  Editor/    ArtLibrary (done) (model → prefab helpers, palette and haze materials)
```

How the pieces connect (M1):

- `BasketInputSource` is the seam for control. `FallbackInput` (keyboard ←/→ A/D, mouse while it's over the frame) implements it today; the face mapper (M3) will be a second implementation, and `BasketController` doesn't care which one it gets.
- `GameStateMachine.Confirm()` / `TogglePause()` are public, so a face gesture can drive them later (Space/Enter and P call them today).
- `GameState.Calibrate` exists but is unused until M3.
- **The scene is now hand-maintained [decided 2026-10-27].** The code scene builder was run one last time and then retired; its source is kept for reference in `Docs/archive/GreyboxSceneBuilder.cs.txt` (outside `Assets`, so it isn't compiled). `Game.unity` is the source of truth: edit it in the Editor, and Claude edits the open scene live and saves it. `ArtLibrary` stays as an editor utility for turning new models into prefabs.
- **OBJ colours:** Unity reads OBJ `Kd` as linear and washes it out, so `ArtLibrary` rebuilds OBJ materials as shared palette materials with the intended sRGB colours (`Art/Materials/Palette/Obj_*`). Tree FBX colours import correctly and are kept.

Scene hierarchy: `Main Camera`, `Directional Light`, `L1_Foreground`, `L2_Play` (Basket, FruitSpawner, Walls), `L3_Person` (quad), `L4_Pile` (floor, bounds, pile root), `L5_Bokeh` (quad), `UI` (Canvas), `Systems` (state machine, webcam, trackers).

---

13. Milestones

#	Goal	Done when
M1 (done)	Greybox loop	The keyboard moves a placeholder basket; fruit spawns one at a time with constant vy, capped vx and wall bounce; catch/miss, per-type score and stats, the 60 s timer, Game Over and restart all work; lives are coded but off. Verified 2026-10-27 in a full Editor round: 28 fruit resolved, catches within the mouth and misses outside it, the round ended on the timer after the last fruit landed, and lives hit 0 without ending the round.

#	Goal	Done when
M1.1 (done)	Playtest 1 fixes	Basket physics collisions, bigger assist, tuning, mouse-only menus (start / pause / main menu / game over), two-column tally. Verified 2026-10-27 in the Editor: rim hits bounce fruit away or occasionally in, every button path works when invoked, and the tally fits. Real mouse clicks still need a manual check (simulated clicks can't reach an unfocused Game view).
M2 (done)	Art pass	Real fruit, trees, bushes, basket model, lighting and the styled score counter; the 3:4 portrait frame is set up. Done 2026-10-27: Low Poly Food fruit (2 variants each, shared atlas), oak canopies in both top corners (the maple was dropped because its red-orange competed with the fruit), recoloured Nature Kit shrubs and flowers along the bottom, our own basket model, a warm sun + trilight ambient, and a global post volume (saturation +10, contrast +6, soft bloom, vignette 0.2, no tonemapping). Fruit enlarged to 0.55 m and the mouth to 1.25 m. A full round ran in the Editor with no errors. L3-L5 (person, pile, bokeh) are still placeholders for M4/M5.
M3a (done)	Webcam in	`WebcamSource` (Systems) picks a camera (`GameConfig.preferredCameraName`, else the first non-built-in one, e.g. an iPhone via Continuity Camera, else any) and keeps a mirrored, centre-cropped 3:4 RenderTexture (540×720). `WebcamBackdrop` shows it, unblurred for now, on the L5 quad (unlit `Webcam_Backdrop` material), with a plain colour fallback if there's no camera or no frames after 3 s. The placeholder person is hidden. Verified 2026-10-27 with an iPhone (Continuity Camera) at 1280×720.

#	Goal	Done when
M3b (done)	Face detection	`FaceTracker` (Systems) runs BlazeFace short-range (`ML/blaze_face_short_range.onnx` + `blaze_face_anchors.csv`, Apache-2.0, from `unity/inference-engine-blaze-face`) on the GPU through Inference Engine. The 3:4 frame is pillarboxed into 128×128; score filtering and NMS are baked into the graph as in Unity's sample. It tracks one face, the largest, but sticks with the previous one while IoU ≥ 0.3, and outputs `HasFace`, `FaceRect`/`FaceCenter` in webcam UV (mirrored, bottom-left origin, matching the screen), `Score` and `LastSeenTime`. `FaceDebugOverlay` (F1) draws a green box plus camera/score/rate status. The backdrop quad was resized to fill the view exactly so the UV lines up with the screen. Verified 2026-10-27 with the iPhone camera: 94 % score, ~130 detections/s, box on the face.
M3c (done)	Face → basket	`FaceToBasketMapper` implements §5's mapping: per-side range from `CalibrationData` (PlayerPrefs; defaults 0.5 / 0.35 / 0.65 until M3d), × `sensitivity`, a rescaled dead zone (0.04), a soft clamp past the 0.8 knee, then a One-Euro filter (min cutoff 1.2, β 0.4), with the basket's 18 m/s cap last. `BasketInputRouter` is the basket's input: the face whenever the webcam is live, else mouse/keyboard; F2 forces mouse/keyboard. `GameStateMachine` pauses with "Can't see you!" after 1 s without a face during a round, and resumes 1 s after the face is back (a manual pause still waits for Resume). Verified 2026-10-27: the mapping maths checks out (u 0.575 → t 0.48 → 0.70 m), the face-lost pause resumed on its own, and a face-controlled round caught 5 out of 5 in its first 15 s.

#	Goal	Done when
M3d (done)	Calibration	`CalibrationController` (Systems) runs look straight → head left → head right; each pose is captured once the face stays within ±`steadyTolerance` (0.015 u) for `steadyHoldTime` (1 s), shown as a pink ring filling around the face. Left and right must clear the centre by `calibMinRangePerSide`, and a face narrower than 12 % or wider than 45 % of the frame gets "move closer / back". Then comes **Try it!** (the basket follows the new range) with **Looks good** (saves to PlayerPrefs) and **Redo**; **Back** / Esc cancels and keeps the old calibration. It runs automatically on the first Play with a live camera and no saved calibration; re-run it with the start menu's **Calibrate** button or C. Verified 2026-10-27: a live player calibrated (L 0.19 / C 0.50 / R 0.82), accepted, and the round started on the new range.
M3 (done) (M3a-d)	Face control	Webcam in, BlazeFace running, mapping + One-Euro filter, the calibration stage, face-lost pause. Playable with the face.
M4	Person & bokeh	The segmentation mask drives the person quad; the blurred bokeh background; the fallbacks.
M5 (done) (pulled forward)	Fruit pile	Caught fruit stacks in the narrow L4 container with every anti-glitch measure; stress-tested with 40+ fruit. Done 2026-10-27, before M3/M4 on request.
M6	Polish	Assist tuning, juice, audio, settings, debug overlay; macOS and Windows builds with camera permission.
M7	Photo booth	After a round: a framed souvenir photo of the player with that round's fruit pile, a Korean-style beauty filter, lab + school logos and the score/best score, saved locally. Design in §15. Needs M4 (person cut-out) and M5 (pile).

---

14. Decisions log

#	Question	Decision
1	End condition	**60 s timer.** A lives counter is coded in but unused, as a fallback (`endCondition` switch).
2	Background pile	A **copy** drops behind the person into a **very narrow invisible rectangular container**, so fruit stacks like 2D but looks 3D. Nothing stacks in the basket.

#	Question	Decision
3	Basket vs. face	The basket **may move away from under the chin**. Crossing the whole shot with your head is unviable, and the game is aimed at **accessibility**. Per-player **calibration** of left/right head range gives the finest control.
4	Fruit variety	All worth the same for now, but **each fruit is tracked as its own type**, so variety-based scoring can come later.
5	Bushes / SFX / basket	**Download CC0 packs first** into `~/Documents/3D Assets/` and **show them immediately** for a yes/no.
6	Platform	**macOS and Windows**; macOS only is acceptable if something can't be made cross-platform.
7	Fruit vs basket (playtest 1)	Fruit get **real physics collisions with the basket**: hitting the rim or a side bangs them away; the assist radius and strength go up so the attractor is noticeable.
8	Tuning (playtest 1)	Faster initial fall (2.7 m/s), wider sideways range (0.7–1.2 m/s) so wall bounces are visible, basket top speed doubled (18 m/s).
9	Booth UI (playtest 1)	**Everything mouse-operable**; the pause menu gets **Main menu**.
10	Game over layout (playtest 1)	Same-size score; a smaller **two-column** fruit tally so long runs don't overflow.
11	Ordering (2026-10-27)	More **UI tweaks are coming from you later**; M2 (art pass) goes first. Keep UI code easy to restyle (GameHUD + builder helpers).
12	Art tweaks (2026-10-27)	Fruit ×1.6 (0.88 m), with the basket scaled up to 1.6 m to match; spin 120–240 °/s; basket tilt 5°; four small foreground trees; hazy light-blue distant trees.
13	Fruit pile	Built now rather than waiting (M5 pulled ahead of M3/M4).
14	Scene builder	Retired; the scene is edited directly and stays visible in the Editor.
15	Pile readability	Pile fruit hazed like the distant trees (65 % toward #A9C6DD), with no shadows, so they read as background and never compete with the falling fruit.
16	Depth cues	Keep the rim light on live fruit and the slight depth-of-field blur on the far layers. The ground shadow was tried and then scrapped completely. A smaller pile and a lower cap were rejected as hurting the look.
17	Photo booth (2026-10-27)	New milestone M7: a photo with the last round's pile, a Korean-style beauty filter, an outline frame with the lab's and school's logos, and the best score (§15).

15. Photo booth mode (M7) [decided 2026-10-27 / details filled in]

A souvenir photo at the end of a round: the player, in front of the fruit pile they just caught, softened with a Korean-style beauty filter, in a branded frame with the lab's and school's logos and their scores.

Flow (all mouse-operable, like every other screen)

1. **Game over → "Take a photo" button** (a third button next to Play again / Main menu).
2. **Pose screen:** the live scene stays up with the round's pile still in place (it isn't cleared until the next round starts). The filter is already applied, so players can see how they'll look, with a "Get ready" prompt and a big **Take photo** button.
3. **3 · 2 · 1 countdown** (the countdown tick), then a white **flash** and a shutter sound.
4. **Preview:** the framed photo is shown frozen, with **Save**, **Retake** and **Done**.
5. **Save** writes a PNG to `~/Pictures/FruitGame/` (Windows: `Pictures\FruitGame\`) named `FruitGame_<date>_<time>_<score>` then returns to the start menu. **Done** returns without saving.

What's in the photo

Element	Spec
Canvas	Portrait **1080 × 1440** (the game's 3:4), rendered offscreen at full resolution regardless of window size
Background	The scene behind the player: the hazed distant trees and **that round's fruit pile, un-hazed for the photo** so the catch shows in full colour, plus the bokeh webcam backdrop
Player	The person cut-out from M4 (segmentation mask) with the beauty filter applied; the basket and falling fruit are hidden
Foreground	The small foreground trees and bushes, for depth
Frame / outline	A rounded white "sticker" border (≈ 40 px) with a thin pink (#C2185B) inner keyline, matching the score counter's style
Logos	**Lab logo** and **school logo** in the bottom band of the frame, left and right, at equal visual height (≈ 90 px). Supplied as PNG/SVG with transparency
Scores	Bottom band, centre: **this round's score** (big, sticker style) and **"Best "** (smaller), plus a small per-fruit icon strip if there's room
Caption	Optional one-line title along the top band (e.g. the event name and date)

Korean-style beauty filter ("뽀샤시" look)

Applied only inside the person mask, then blended back, so the scene isn't touched:

1. **Skin smoothing:** an edge-preserving blur (bilateral or guided filter) at ~50–60 % blend, restricted to a soft skin mask (YCrCb skin range ∩ person mask, eroded away from the eyes and mouth using BlazeFace's six keypoints).
2. **Brightening + soft glow:** lift exposure by about 0.2 EV, lower contrast slightly, and add a gentle bloom ("soft focus") on highlights.
3. **Tone:** a slight warm-pink shift in the midtones and a touch of desaturation in the shadows; a clean, airy palette.
4. **Blush:** two very soft pink radial gradients on the cheeks, placed from the eye and mouth keypoints, at about 15 % opacity.
5. **Optional, off by default:** subtle eye enlargement / face slimming (a mesh warp from keypoints). Some people find this uncomfortable, so it stays off unless you want it.

All of it runs on the GPU as a few full-screen passes on the captured frame, so there's no extra ML model beyond M3's BlazeFace and M4's segmentation. Strength is one slider (0 = off) in `GameConfig`.

Privacy (booth use)

- Photos are only saved when the player presses **Save**; nothing is uploaded anywhere.
- A small "Photos are saved on this computer" note sits on the preview screen.
- An optional `photoAutoDeleteDays` setting cleans old photos out of the folder at startup.

Open questions for M7

1. **Logo files:** please provide the lab and school logos (PNG with transparency or SVG), plus the exact names/wording to show, if any.
2. **Getting photos to people:** save only? Or also a **QR code** to download (needs a server or local Wi-Fi hosting), **printing**, or **AirDrop**?
3. **Caption:** an event name/date along the top band?
4. **Fruit pile in the photo:** full-colour (current plan) or keep the in-game haze?